

assignment-06-document-classification

April 5, 2026

0.1 Assignment 6 Instructions: Document Classification

It can be useful to be able to classify new “test” documents using already classified “training” documents. A common example is using a corpus of labeled spam and ham (non-spam) e-mails to predict whether or not a new document is spam. Here is one example of such data: UCI Machine Learning Repository: Spambase Data Set

For this project, you can either use the above dataset to predict the class of new documents (either withheld from the training dataset or from another source such as your own spam folder).

For more adventurous students, you are welcome (encouraged!) to come up a different set of documents (including scraped web pages!?) that have already been classified (e.g. tagged), then analyze these documents to predict how new documents should be classified.

0.1.1 Executive Summary Draft

This project builds a document classification model to distinguish between spam and non-spam emails using the **(UCI Spambase dataset)**[<https://archive.ics.uci.edu/dataset/94/spambase>]. Following techniques from the NLTK book, and modified using the scikit-learn machine learning library, I preprocess text-like features, train a classifier (Naive Bayes), and evaluate its performance using common classification metrics.

The model successfully learned patterns associated with spam emails and achieved an overall accuracy of 82% on unseen test data. It demonstrated strong recall performance in detecting spam (0.95), correctly identifying 369 out of 390 spam emails, indicating a low rate of missed spam. However, the model had a lower precision for spam classification (0.72), incorrectly classifying 143 legitimate emails as spam (false positives). The model also achieved an ROC-AUC score of 0.94, indicating a very good overall ability to distinguish between spam and non-spam across different decision thresholds.

Attempts to improve performance by adjusting the classification threshold from 0.5 to .70 had minimal impact, suggesting that the observed false positive rate is driven more by model limitations than decision boundary settings. Overall, the model performs well at identifying spam but tends to aggressively classify emails as spam.

Module 1: Download spam dataset directly from the UCI Online Repository The UCI collection of spam e-mails came from their postmaster and individuals who had filed spam. Their collection of non-spam e-mails came from filed work and personal e-mails, and hence the word ‘george’ and the area code ‘650’ are indicators of non-spam. Per the authors of the dataset, typical performance is around ~7% misclassification error. False positives (marking good mail as spam) are

very undesirable. If zero false positives are required in the training/testing set, 20-25% of the spam passed through the filter. The authors note that there are missing values in the dataset, which I will need to address.

```
[1]: from ucimlrepo import fetch_ucirepo
```

```
# fetch dataset
spambase = fetch_ucirepo(id=94)

# data (as pandas dataframes)
X = spambase.data.features
y = spambase.data.targets

# metadata
print(spambase.metadata)

# variable information
print(spambase.variables)
```

```
{'uci_id': 94, 'name': 'Spambase', 'repository_url':
'https://archive.ics.uci.edu/dataset/94/spambase', 'data_url':
'https://archive.ics.uci.edu/static/public/94/data.csv', 'abstract':
'Classifying Email as Spam or Non-Spam', 'area': 'Computer Science', 'tasks':
['Classification'], 'characteristics': ['Multivariate'], 'num_instances': 4601,
'num_features': 57, 'feature_types': ['Integer', 'Real'], 'demographics': [],
'target_col': ['Class'], 'index_col': None, 'has_missing_values': 'no',
'missing_values_symbol': None, 'year_of_dataset_creation': 1999, 'last_updated':
'Mon Aug 28 2023', 'dataset_doi': '10.24432/C53G6X', 'creators': ['Mark
Hopkins', 'Erik Reeber', 'George Forman', 'Jaap Suermondt'], 'intro_paper':
None, 'additional_info': {'summary': 'The "spam" concept is diverse:
advertisements for products/web sites, make money fast schemes, chain letters,
pornography...\n\nThe classification task for this dataset is to determine
whether a given email is spam or not.\n\t\nOur collection of spam e-mails came
from our postmaster and individuals who had filed spam. Our collection of non-
spam e-mails came from filed work and personal e-mails, and hence the word
\'george\' and the area code \'650\' are indicators of non-spam. These are
useful when constructing a personalized spam filter. One would either have to
blind such non-spam indicators or get a very wide collection of non-spam to
generate a general purpose spam filter.\n\nFor background on spam: Cranor,
Lorrie F., LaMacchia, Brian A. Spam!, Communications of the ACM, 41(8):74-83,
1998.\n\nTypical performance is around ~7% misclassification error. False
positives (marking good mail as spam) are very undesirable.If we insist on zero
false positives in the training/testing set, 20-25% of the spam passed through
the filter. See also Hewlett-Packard Internal-only Technical Report. External
version forthcoming. ', 'purpose': None, 'funded_by': None,
'instances_represent': 'Emails', 'recommended_data_splits': None,
'sensitive_data': None, 'preprocessing_description': None, 'variable_info': 'The
last column of \'spambase.data\' denotes whether the e-mail was considered spam
```

(1) or not (0), i.e. unsolicited commercial e-mail. Most of the attributes indicate whether a particular word or character was frequently occurring in the e-mail. The run-length attributes (55-57) measure the length of sequences of consecutive capital letters. For the statistical measures of each attribute, see the end of this file. Here are the definitions of the attributes:\r\n\r\n48 continuous real [0,100] attributes of type word_freq_WORD \r\n= percentage of words in the e-mail that match WORD, i.e. $100 * (\text{number of times the WORD appears in the e-mail}) / \text{total number of words in e-mail}$. A "word" in this case is any string of alphanumeric characters bounded by non-alphanumeric characters or end-of-string.\r\n\r\n6 continuous real [0,100] attributes of type char_freq_CHAR \r\n= percentage of characters in the e-mail that match CHAR, i.e. $100 * (\text{number of CHAR occurrences}) / \text{total characters in e-mail}$ \r\n\r\n1 continuous real [1,...] attribute of type capital_run_length_average \r\n= average length of uninterrupted sequences of capital letters\r\n\r\n1 continuous integer [1,...] attribute of type capital_run_length_longest \r\n= length of longest uninterrupted sequence of capital letters\r\n\r\n1 continuous integer [1,...] attribute of type capital_run_length_total \r\n= sum of length of uninterrupted sequences of capital letters \r\n= total number of capital letters in the e-mail\r\n\r\n1 nominal {0,1} class attribute of type spam\r\n= denotes whether the e-mail was considered spam (1) or not (0), i.e. unsolicited commercial e-mail. \r\n\r\n', 'citation': None}}

	name	role	type	demographic \
0	word_freq_make	Feature	Continuous	None
1	word_freq_address	Feature	Continuous	None
2	word_freq_all	Feature	Continuous	None
3	word_freq_3d	Feature	Continuous	None
4	word_freq_our	Feature	Continuous	None
5	word_freq_over	Feature	Continuous	None
6	word_freq_remove	Feature	Continuous	None
7	word_freq_internet	Feature	Continuous	None
8	word_freq_order	Feature	Continuous	None
9	word_freq_mail	Feature	Continuous	None
10	word_freq_receive	Feature	Continuous	None
11	word_freq_will	Feature	Continuous	None
12	word_freq_people	Feature	Continuous	None
13	word_freq_report	Feature	Continuous	None
14	word_freq_addresses	Feature	Continuous	None
15	word_freq_free	Feature	Continuous	None
16	word_freq_business	Feature	Continuous	None
17	word_freq_email	Feature	Continuous	None
18	word_freq_you	Feature	Continuous	None
19	word_freq_credit	Feature	Continuous	None
20	word_freq_your	Feature	Continuous	None
21	word_freq_font	Feature	Continuous	None
22	word_freq_000	Feature	Continuous	None
23	word_freq_money	Feature	Continuous	None
24	word_freq_hp	Feature	Continuous	None
25	word_freq_hpl	Feature	Continuous	None

26	word_freq_george	Feature	Continuous	None
27	word_freq_650	Feature	Continuous	None
28	word_freq_lab	Feature	Continuous	None
29	word_freq_labs	Feature	Continuous	None
30	word_freq_telnet	Feature	Continuous	None
31	word_freq_857	Feature	Continuous	None
32	word_freq_data	Feature	Continuous	None
33	word_freq_415	Feature	Continuous	None
34	word_freq_85	Feature	Continuous	None
35	word_freq_technology	Feature	Continuous	None
36	word_freq_1999	Feature	Continuous	None
37	word_freq_parts	Feature	Continuous	None
38	word_freq_pm	Feature	Continuous	None
39	word_freq_direct	Feature	Continuous	None
40	word_freq_cs	Feature	Continuous	None
41	word_freq_meeting	Feature	Continuous	None
42	word_freq_original	Feature	Continuous	None
43	word_freq_project	Feature	Continuous	None
44	word_freq_re	Feature	Continuous	None
45	word_freq_edu	Feature	Continuous	None
46	word_freq_table	Feature	Continuous	None
47	word_freq_conference	Feature	Continuous	None
48	char_freq_;	Feature	Continuous	None
49	char_freq_(	Feature	Continuous	None
50	char_freq_[	Feature	Continuous	None
51	char_freq_!	Feature	Continuous	None
52	char_freq_\$	Feature	Continuous	None
53	char_freq_#	Feature	Continuous	None
54	capital_run_length_average	Feature	Continuous	None
55	capital_run_length_longest	Feature	Continuous	None
56	capital_run_length_total	Feature	Continuous	None
57	Class	Target	Binary	None

	description	units	missing_values
0	None	None	no
1	None	None	no
2	None	None	no
3	None	None	no
4	None	None	no
5	None	None	no
6	None	None	no
7	None	None	no
8	None	None	no
9	None	None	no
10	None	None	no
11	None	None	no
12	None	None	no
13	None	None	no

14	None	None	no
15	None	None	no
16	None	None	no
17	None	None	no
18	None	None	no
19	None	None	no
20	None	None	no
21	None	None	no
22	None	None	no
23	None	None	no
24	None	None	no
25	None	None	no
26	None	None	no
27	None	None	no
28	None	None	no
29	None	None	no
30	None	None	no
31	None	None	no
32	None	None	no
33	None	None	no
34	None	None	no
35	None	None	no
36	None	None	no
37	None	None	no
38	None	None	no
39	None	None	no
40	None	None	no
41	None	None	no
42	None	None	no
43	None	None	no
44	None	None	no
45	None	None	no
46	None	None	no
47	None	None	no
48	None	None	no
49	None	None	no
50	None	None	no
51	None	None	no
52	None	None	no
53	None	None	no
54	None	None	no
55	None	None	no
56	None	None	no
57	spam (1) or not spam (0)	None	no

Module 2: Combine Labels and Inspect the Spambase Data In this module, I combined the categories and the labels into one dataframe and inspected the dataframe.

```
[2]: import pandas as pd

# Combine X and y into one DataFrame
y.columns = ["label"]
df = pd.concat([X, y], axis=1)
# df.to_csv("spambase.csv", index=False)
```

```
[4]: df.head()
```

```
[4]: word_freq_make word_freq_address word_freq_all word_freq_3d \
0          0.00          0.64          0.64          0.0
1          0.21          0.28          0.50          0.0
2          0.06          0.00          0.71          0.0
3          0.00          0.00          0.00          0.0
4          0.00          0.00          0.00          0.0

word_freq_our word_freq_over word_freq_remove word_freq_internet \
0          0.32          0.00          0.00          0.00
1          0.14          0.28          0.21          0.07
2          1.23          0.19          0.19          0.12
3          0.63          0.00          0.31          0.63
4          0.63          0.00          0.31          0.63

word_freq_order word_freq_mail ... char_freq_ char_freq_ ( \
0          0.00          0.00 ...          0.00          0.000
1          0.00          0.94 ...          0.00          0.132
2          0.64          0.25 ...          0.01          0.143
3          0.31          0.63 ...          0.00          0.137
4          0.31          0.63 ...          0.00          0.135

char_freq_ char_freq_! char_freq_$ char_freq_# \
0          0.0          0.778          0.000          0.000
1          0.0          0.372          0.180          0.048
2          0.0          0.276          0.184          0.010
3          0.0          0.137          0.000          0.000
4          0.0          0.135          0.000          0.000

capital_run_length_average capital_run_length_longest \
0          3.756          61
1          5.114          101
2          9.821          485
3          3.537          40
4          3.537          40

capital_run_length_total label
0          278          1
1          1028          1
```

2	2259	1
3	191	1
4	191	1

[5 rows x 58 columns]

Module 3: Look for Missing Values In the dataset information, the authors noted that it contains missing values. Although the authors state that the dataset contains missing values, I see none.

```
[5]: # Check total missing values in each column
missing_per_column = X.isnull().sum()

print(missing_per_column)
```

```
word_freq_make          0
word_freq_address       0
word_freq_all           0
word_freq_3d            0
word_freq_our           0
word_freq_over          0
word_freq_remove        0
word_freq_internet      0
word_freq_order         0
word_freq_mail          0
word_freq_receive       0
word_freq_will          0
word_freq_people        0
word_freq_report        0
word_freq_addresses     0
word_freq_free          0
word_freq_business      0
word_freq_email         0
word_freq_you           0
word_freq_credit        0
word_freq_your          0
word_freq_font          0
word_freq_000           0
word_freq_money         0
word_freq_hp            0
word_freq_hpl           0
word_freq_george        0
word_freq_650           0
word_freq_lab           0
word_freq_labs          0
word_freq_telnet        0
word_freq_857           0
word_freq_data          0
```

```

word_freq_415      0
word_freq_85       0
word_freq_technology  0
word_freq_1999     0
word_freq_parts    0
word_freq_pm       0
word_freq_direct   0
word_freq_cs       0
word_freq_meeting  0
word_freq_original 0
word_freq_project  0
word_freq_re       0
word_freq_edu      0
word_freq_table    0
word_freq_conference 0
char_freq_;        0
char_freq_(        0
char_freq_[        0
char_freq_!        0
char_freq_$       0
char_freq_#        0
capital_run_length_average  0
capital_run_length_longest  0
capital_run_length_total    0
dtype: int64

```

Module 4: Check for Class Imbalance In this module, to prevent a biased classifier, I checked for class imbalance to make sure that I have appropriate representation between spam and nonspam emails. The classes are slightly imbalanced, but still acceptable without further treatment for modeling.

```

[6]: # Percentage in each class
class_percent = (y.value_counts(normalize=True) * 100).round(2)

print(class_percent)

```

```

label
0      60.6
1      39.4
Name: proportion, dtype: float64

```

Module 5: Split into Training and Test Sets Per Chapter 6 in Bird et al. (2009), the nltk library classifier is not designed for dataframes. To use it, classification has to be in the form of a feature set, label pairing. The spambase data is currently in a pandas dataframe, and the nltk classifier does not work well with it. The more direct way to create a workflow is to use the scikit-learn library, which accepts a pandas dataframe.

At this stage, given that the spambase dataset is already in numeric form, there are no missing

values, they are all int64 datatypes, and the classes are fairly balanced, I divided the dataset into training and testing sets.

```
[7]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y.values.ravel(), test_size=0.2, random_state=42
)

print("Train size:", len(X_train))
print("Test size:", len(X_test))
```

Train size: 3680

Test size: 921

Module 6: Train Naive Bayes Classifier This code sets up and trains a spam detection model. From the scikit-learn library, I am pulling out the Naive Bayes model designed for numeric data. GaussianNB is a specific classifier used when the data consists of numerical values.

The “model” object creates an empty model ready to learn something. The `model.fit(X_train, y_train)` populates the model with which patterns, such as word frequencies or symbols from the training data, are associated with spam versus normal emails. After this step, the model is ready to make predictions on unseen emails.

```
[8]: from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
```

[8]: GaussianNB()

Module 7: Make Predictions The ‘`from sklearn.metrics import accuracy_score, classification_report`’ line imports tools to measure how well the model is performing.

The `y_pred = model.predict(X_test)` line uses the trained model to make predictions on unseen data.

The `predict()` function takes the test data and predicts whether each email is spam, and `accuracy_score` and `classification_report` compare those predictions to the actual labels. It also shows how accurate the model by measuring precision, recall, f1-score, and support to figure out where the model might be making mistakes. The `accuracy_score` informs us how often the model is correct. The precision score informs us how accurate spam predictions are. The recall score informs us how much spam is caught. The F1 score informs us of the balance between precision and recall.

The model achieves 82% accuracy, correctly identifying most emails overall. It is strong at catching spam with 95% recall, but has lower precision at 72% for spam, meaning some legitimate emails are incorrectly classified as spam.

```
[9]: from sklearn.metrics import accuracy_score, classification_report
```

```

y_pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

Accuracy: 0.82

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.73	0.82	531
1	0.72	0.95	0.82	390
accuracy			0.82	921
macro avg	0.83	0.84	0.82	921
weighted avg	0.85	0.82	0.82	921

Module 8: Create and Plot a Confusion Matrix The module analyzes a confusion matrix. Out of 921 emails, the model correctly identified 387 non-spam and 369 spam emails, but incorrectly labeled 144 legitimate emails as spam (false positives) and missed only 21 spam emails (false negatives). This shows that the model is a strong spam detector but over labels normal emails as spam.

```

[10]: from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print(cm)

import matplotlib.pyplot as plt

plt.imshow(cm)
plt.title("Confusion Matrix")
plt.colorbar()

plt.xlabel("Predicted")
plt.ylabel("Actual")

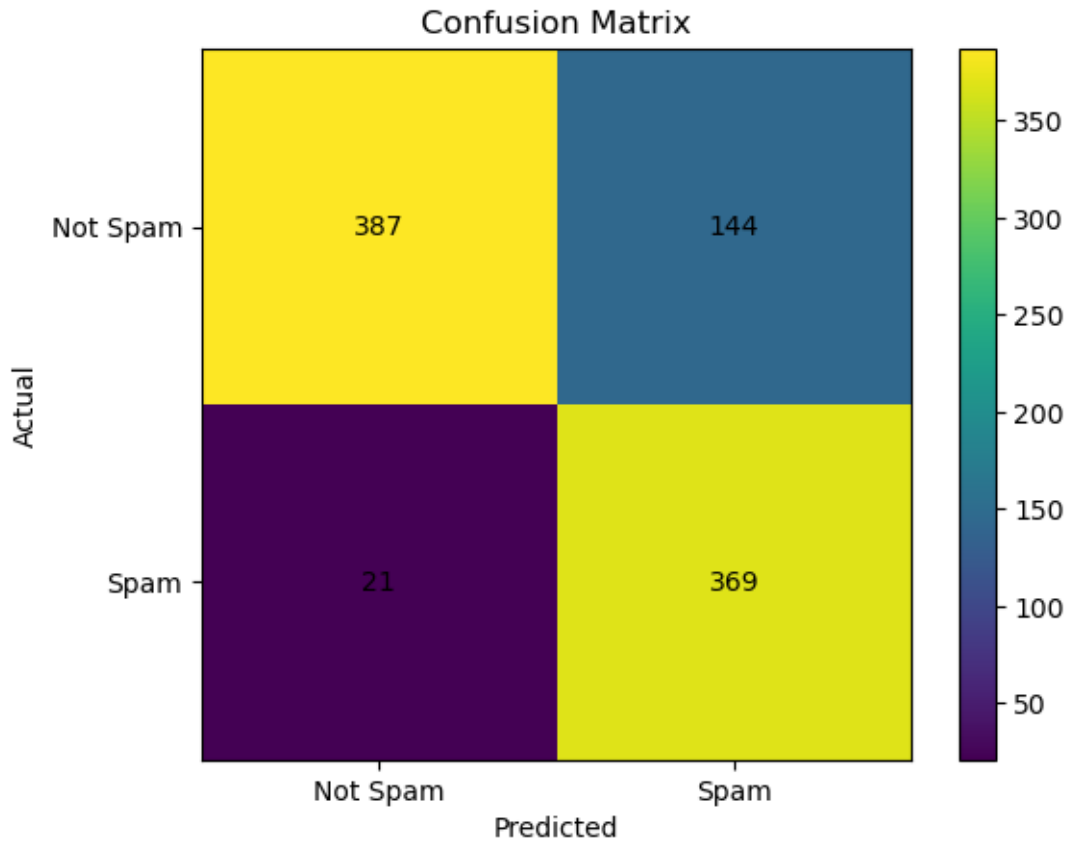
plt.xticks([0,1], ["Not Spam", "Spam"])
plt.yticks([0,1], ["Not Spam", "Spam"])

for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.show()

```

```
[[387 144]
 [ 21 369]]
```



Module 9: ROC Curve The model achieves an AUC score of 0.94, indicating a very good ability to distinguish between spam and non-spam across all thresholds, demonstrating strong overall classification capability.

```
[11]: y_prob = model.predict_proba(X_test)[: , 1]

from sklearn.metrics import roc_curve, roc_auc_score

fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = roc_auc_score(y_test, y_prob)

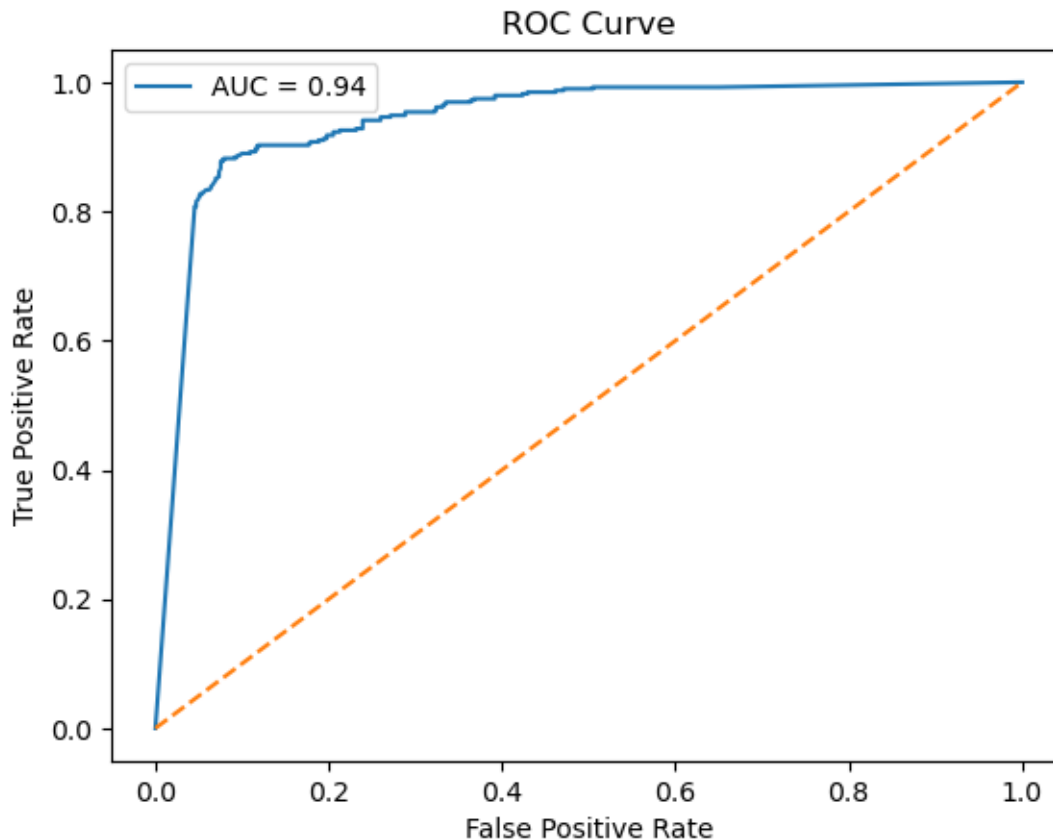
import matplotlib.pyplot as plt

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0,1], [0,1], linestyle="--")

plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()

plt.show()
```



Module 10: Adjustments to Improve False Positives Since the false positive figure was high, I tried increasing the threshold from .05 to 0.7 and reran the model. However, it resulted in no meaningful change in performance. The accuracy level still remained at 0.82, with 389 non-spam and 369 spam emails correctly classified, 142 false positives, and 21 false negatives. Increasing the threshold only improved false positives by 2 emails. This may indicate that a false positive problem may be inherent to the model rather than the decision threshold. However, the model is very strong at detecting spam, as the recall rate is still 0.95.

```
[12]: # Adjust threshold to reduce false positives

y_prob = model.predict_proba(X_test)[: , 1]

threshold = 0.7
```

```

y_pred_adjusted = (y_prob > threshold).astype(int)

print("Threshold:", threshold)
print("Accuracy:", round(accuracy_score(y_test, y_pred_adjusted), 2))
print("\nClassification Report:\n", classification_report(y_test,
    ↪ y_pred_adjusted))

cm = confusion_matrix(y_test, y_pred_adjusted)
print("\nConfusion Matrix:\n", cm)

```

Threshold: 0.7

Accuracy: 0.82

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.73	0.83	531
1	0.72	0.95	0.82	390
accuracy			0.82	921
macro avg	0.84	0.84	0.82	921
weighted avg	0.85	0.82	0.82	921

Confusion Matrix:

```

[[389 142]
 [ 21 369]]

```